upGrad

#LifeKoKaroLift

PG Diploma in ML/AI

upGrad

Course : Machine Learning
Lecture On : NLP
Instructor : Manish Kumar

upGrad

Natural Language Processing

LEXICAL PROCESSING

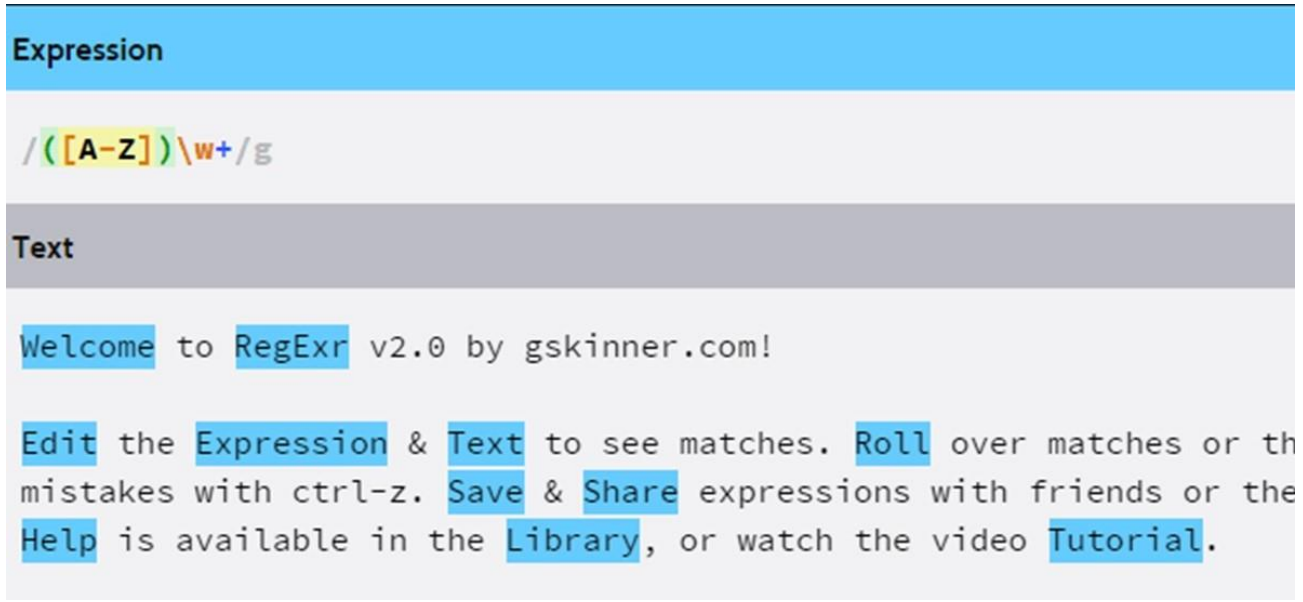
REGEX

upGrad

- Regular expressions (regex or regexp) are extremely useful in extracting information from any text by searching for one or more matches of a specific search pattern

- Fields of application range from validation to parsing/replacing strings, passing through translating data to other formats and web scraping

- The power of regular expressions is that they can specify patterns, not just fixed characters.



Common Regex Patterns

^	Matches the beginning of a line
$	Matches the end of the line
.	Matches any character
\s	Matches whitespace
\S	Matches any non-whitespace character
*	Repeats a character zero or more times
*?	Repeats a character zero or more times (non-greedy)
+	Repeats a character one or more times
+?	Repeats a character one or more times (non-greedy)
[aeiou]	Matches a single character in the listed set
[^XYZ]	Matches a single character not in the listed set
[a-z0-9]	The set of characters can include a range
(	Indicates where string extraction is to start
)	Indicates where string extraction is to end

BAG OF WORDS

upGrad

- A bag-of-words model, or BOW for short, is a way of extracting features from text for use in modeling, such as with machine learning algorithms.
- The approach is very simple and flexible, and can be used in a myriad of ways for extracting features from documents.
- A bag-of-words is a representation of text that describes the occurrence of words within a document.

Example

Sentence 1 : It was the best of times.
Sentence 2 : It was the worst of times.

- Unique Words (Bag of words) = [“it”, “was”, “the”, “best”, “of”, “times”, “worst”]

- Representations:

  - "it was the best of times" = [1,1,1,1,1,1,0]
  - "it was the worst of times" =[1,1,1,0,1,1,1]

Drawbacks of BOW

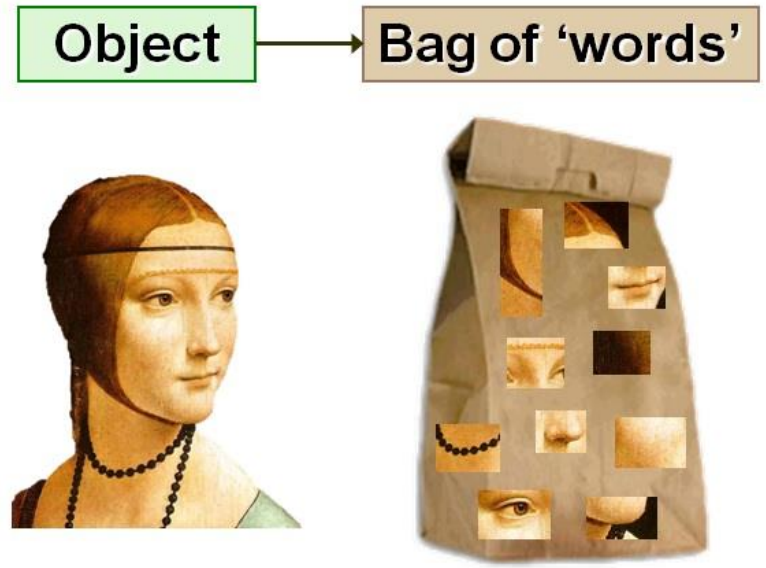
upGrad

In the above example, we can have vectors of length 11. However, we start facing issues when we come across new sentences:

1. If the new sentences contain new words, then our vocabulary size would increase and thereby, the length of the vectors would increase too.

2. Additionally, the vectors would also contain many 0s, thereby resulting in a sparse matrix (which is what we would like to avoid)

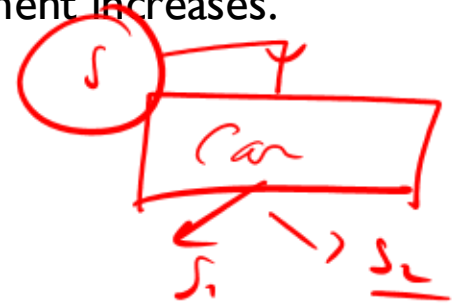
3. We are retaining no information on the grammar of the sentences nor on the ordering of the words in the text.



TFIDF

upGrad

- TF-IDF stands for "Term Frequency—Inverse Document Frequency"
  - It gives us the frequency of the word in each document in the corpus.
  - It is the ratio of number of times the word appears in a document compared to the total number of words in that document.
  - It increases as the number of occurrences of that word within the document increases.
  - Each document has its own term frequency.



Example

Sentence 1 : The car is driven on the road.
Sentence 2 : The truck is driven on the highway.

$$w_{i,j} = tf_{i,j} \times \log\left(\frac{N}{df_i}\right)$$

tf_{ij} = number of occurrences of i in j
df_i = number of documents containing i
N = total number of documents

Word	TF		IDF	TF*IDF	
	A	B		A	B
The	1/7	1/7	log(2/2) = 0	0	0
Car	1/7	0	log(2/1) = 0.3	0.043	0
Truck	0	1/7	log(2/1) = 0.3	0	0.043
Is	1/7	1/7	log(2/2) = 0	0	0
Driven	1/7	1/7	log(2/2) = 0	0	0
On	1/7	1/7	log(2/2) = 0	0	0
The	1/7	1/7	log(2/2) = 0	0	0
Road	1/7	0	log(2/1) = 0.3	0.043	0
Highway	0	1/7	log(2/1) = 0.3	0	0.043

STEMMING

upGrad

- Stemming, in literal terms, is the process of cutting down the branches of a tree to its stem.
  - Language dependent
  - e.g., ***automate(s), automatic, automation*** all reduced to ***automat***.

- Stemmer operates on a single word without knowledge of the context, and therefore cannot discriminate between words which have different meanings depending on part of speech.

- However, stemmers are typically easier to implement and run faster, and the reduced accuracy may not matter for some applications.

- Porter's Algorithm
  - rule based stemmer
  - produces stems, makes a number of mistakes

- Krovetz Algorithm
  - algorithm + dictionary
  - if not found → strip suffix, lookup again
  - produces "words" not stems

LEMMATIZATION

upGrad

- Lemmatization is closely related to stemming.
- In linguistics, it is the process of grouping together the different inflected forms of a word so they can be analyzed as a single item.
- Lemmatization changes the verb form, while keeping the meaning of the word the same.
- Stemming will reduce "Worker" and "Speaker" to "Work" and "Speak" respectively while lemmatization will reduce them to "Worker" and "Speaker" because "Worker" is not an inflected form of "Work", neither is "Speaker" an inflected form of "Speak".

Original	Stemmed	Lemmatized
visibilities	visibl	visibility
adhere	adher	adhere
adhesion	adhes	adhesion
appendicitis	append	appendicitis
oxen	oxen	ox
indices	indic	index
swum	swum	swim

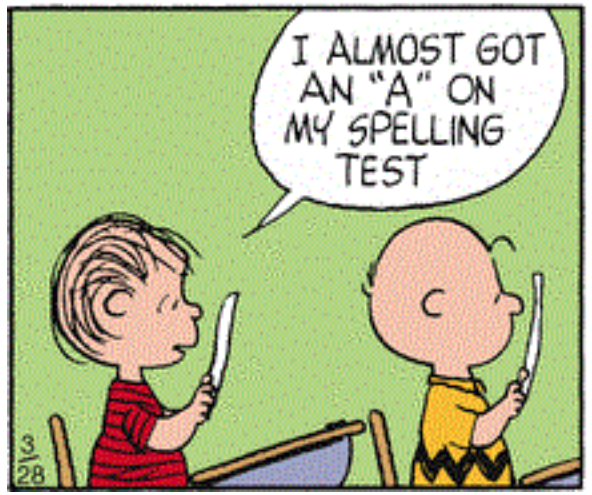
Why Phonetic Hashing ?

upGrad

There are some cases that can't be handled either by stemming nor lemmatization. You need another preprocessing method in order to stem or lemmatize the words efficiently.

Example,

'disappearing' : 'dissappearng' & 'dissapearing'.



After you stem these words, you'll have two different stems 'dissappear' and 'dissapear'.

You still have the problem of redundant tokens.
lemmatization won't even work because it only works on correct dictionary spelling

Phonetic Hashing

upGrad

Phonetic hashing buckets all the similar phonemes (words with similar sound or pronunciation) into a single bucket and gives all these variations a single hash code. Hence, the word 'Dilli' and 'Delhi' will have the same code.

A commonly used conversion results in a 4-character code, with the first character being a letter of the alphabet and the other three being digits between 0 and 9.

1.Retain the first letter of the term.
2.Change all occurrences of the following letters to '0' (zero): 'A', E', 'I', 'O', 'U', 'H', 'W', 'Y'.
3.Change letters to digits as follows:

- B, F, P, V to 1.
- C, G, J, K, Q, S, X, Z to 2.
- D,T to 3.
- L to 4.
- M, N to 5.
- R to 6.
- H,W,Y to unencoded

4.Repeatedly remove one out of each pair of consecutive identical digits.
5.Remove all zeros from the resulting string. Pad the resulting string with trailing zeros and return the first four positions, which will consist of a letter followed by three digits.

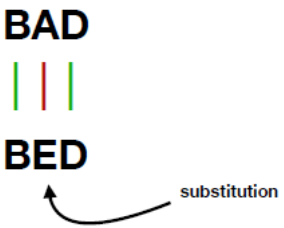
Edit Distance

upGrad

This method was invented by Vladimir Levenshtein in 1965. The value refer to the minimum number of action (deletion, insertion and substitution) required to transform from one word to another word.

Simply put, edit distance is a measurement of how many changes we must do to one string to transform it into the string we are comparing it to.

As an illustration, the difference between “Frederic” and “Fred” is four, as we can change “Frederic” into “Fred” with the deletion of the letters “e” , “r”, “i” and ”c”.



$$\operatorname{lev}_{a,b}(i,j) = \begin{cases} \max(i,j) & \text{if } \min(i,j) = 0, \\ \min \begin{cases} \operatorname{lev}_{a,b}(i-1,j) + 1 \\ \operatorname{lev}_{a,b}(i,j-1) + 1 \\ \operatorname{lev}_{a,b}(i-1,j-1) + 1_{(a_i \neq b_j)} \end{cases} & \text{otherwise.} \end{cases}$$

https://en.wikipedia.org/wiki/Levenshtein_distance

To calculate the Levenshtein Distance, we have to go through every single word and using the minimum + 1 from

- (i-1, j): It means left box (deletion)
- (i, j-1): It means upper box (insertion)
- (i-1, j-1): It means left upper box (substitution)

Example

upGrad

Step 1

		e	d	w	a	r	d
	0	1	2	3	4	5	6
e	1						
d	2						
w	3						
i	4						
n	5						

Step 2

		e	d	w	a	r	d
	0	1	2	3	4	5	6
e	1	0					
d	2						
w	3						
i	4						
n	5						

Step 3

		e	d	w	a	r	d
	0	1	2	3	4	5	6
e	1	0	1				
d	2						
w	3						
i	4						
n	5						

Step 4

		e	d	w	a	r	d
	0	1	2	3	4	5	6
e	1	0	1	2	3	4	5
d	2						
w	3						
i	4						
n	5						

Step 5

		e	d	w	a	r	d
	0	1	2	3	4	5	6
e	1	0	1	2	3	4	5
d	2	1					
w	3						
i	4						
n	5						

Step 6

		e	d	w	a	r	d
	0	1	2	3	4	5	6
e	1	0	1	2	3	4	5
d	2	1	0				
w	3						
i	4						
n	5						

Step 7

		e	d	w	a	r	d
	0	1	2	3	4	5	6
e	1	0	1	2	3	4	5
d	2	1	0	1			
w	3						
i	4						
n	5						

Step 8

		e	d	w	a	r	d
	0	1	2	3	4	5	6
e	1	0	1	2	3	4	5
d	2	1	0	1	2	3	4
w	3	2	1	0	1	2	3
i	4	3	2	1	1	2	3
n	5	4	3	2	2	2	3

Step 9

		e	d	w	a	r	d
	0	1	2	3	4	5	6
e	1	0	1	2	3	4	5
d	2	1	0	1	2	3	4
w	3	2	1	0	1	2	3
i	4	3	2	1	1	2	3
n	5	4	3	2	2	2	3

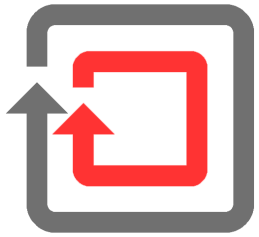
Step 10

		e	d	w	a	r	d
	0	1	2	3	4	5	6
e	1	0	1	2	3	4	5
d	2	1	0	1	2	3	4
w	3	2	1	0	1	2	3
i	4	3	2	1	1	2	3
n	5	4	3	2	2	2	3

1.Substitute “n” by “d”

2.Substitute “i” by “r”

3.Delete “a”

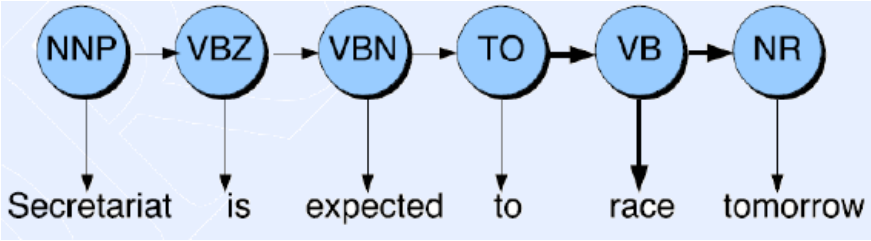


SYNTACTIC PROCESSING

POS - TAGGING

upGrad

Let's say we ask Alexa a question - "Ok Alexa, where can I get the permit to work in Australia?". Now, the word **'permit'** can potentially have two POS tags - noun and a verb. In the phrase 'I need a work permit', the correct tag of **'permit'** is **'noun'**. On the other hand, in the phrase "Please permit me to take the exam.", the word **'permit'** is a **'verb'**.



Assigning the correct POS tags helps us better understand the intended meaning of a phrase or a sentence and is thus a crucial part of syntactic processing

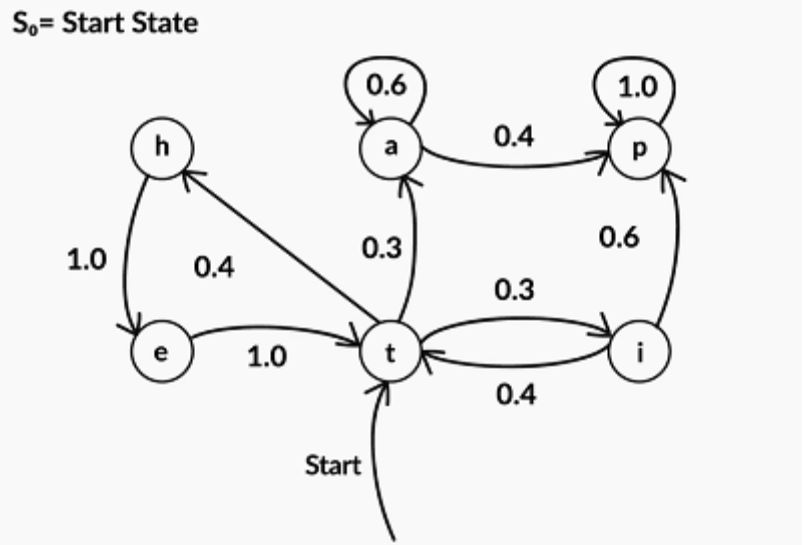
Some well known Use-cases:

- text-to-speech (how do we pronounce "lead"?)
- Helps in writing better regexps like (Det) Adj* N+ over the output etc.
- Is generally included as a feature in Named Entity Recognition models

upGrad

Markov Chain

Markov models are probabilistic models that were developed to model sequential processes.
A Markov chain is used to represent a process which performs a transition from one state to other. This transition makes an assumption that the probability of transitioning to the next state is dependent solely on the current state.

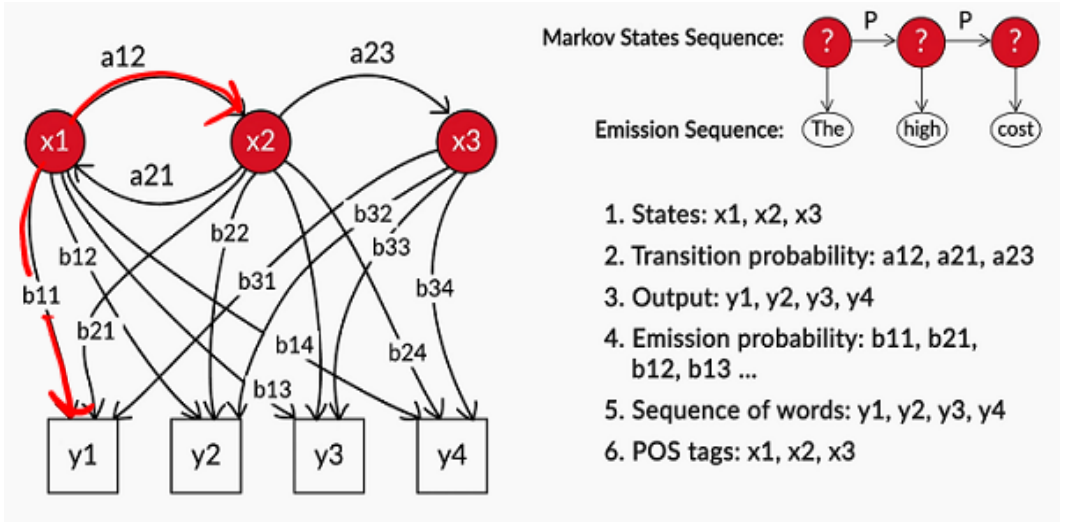


Here, 'a', 'p', 'i', 't', 'e', 'h' are the **states** and the numbers mentioned on the edges are **transition probabilities.**

Hidden Markov Model

The **Hidden Markov Model** (HMM) is an extension to the Markov process which is used to model phenomena where the **states are hidden** and they **emit observations**.

For Example, in POS tagging, what you observe are the words in a sentence, while the POS tags themselves are hidden. Thus, you can model the POS tagging task as an HMM with the hidden states representing POS tags which emit observations, i.e. words.



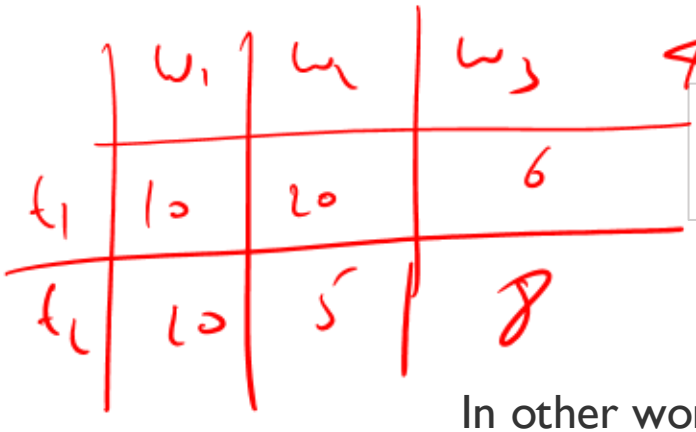
The hidden states **emit observations** with a certain probability. Therefore, along with the transition and initial state probabilities, Hidden Markov Models also have **emission probabilities** which represent the probability that an observation is emitted by a particular state.

POS - TAGGING

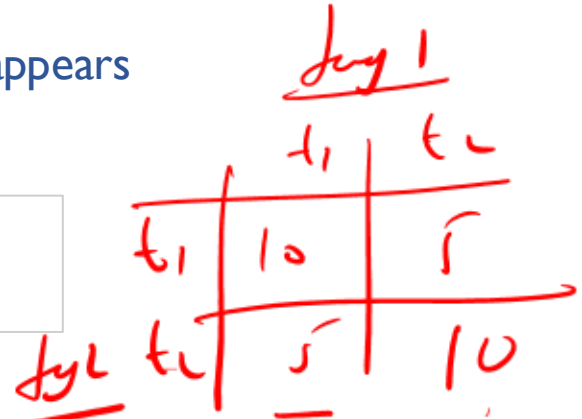
upGrad

Emission Probability of a word 'w' for tag 't':
P(w|t) = Number of times w has been tagged t/Number of times t appears

Transition Probability of tag t1 followed by tag t2:
P(t2|t1) = Number of times t1 is followed by tag t2/ Number of times t1 appears



P(tag|word) = P(word|tag) * P(tag|previous tag)
= Emission probability * Transition probability



In other words, we assign the tag t to the word w which has the max P(tag|word).

$P(w_2/t_1) = (20/36)$

$P(t_2/t_1) = (5/15) = 1/3$

Viterbi

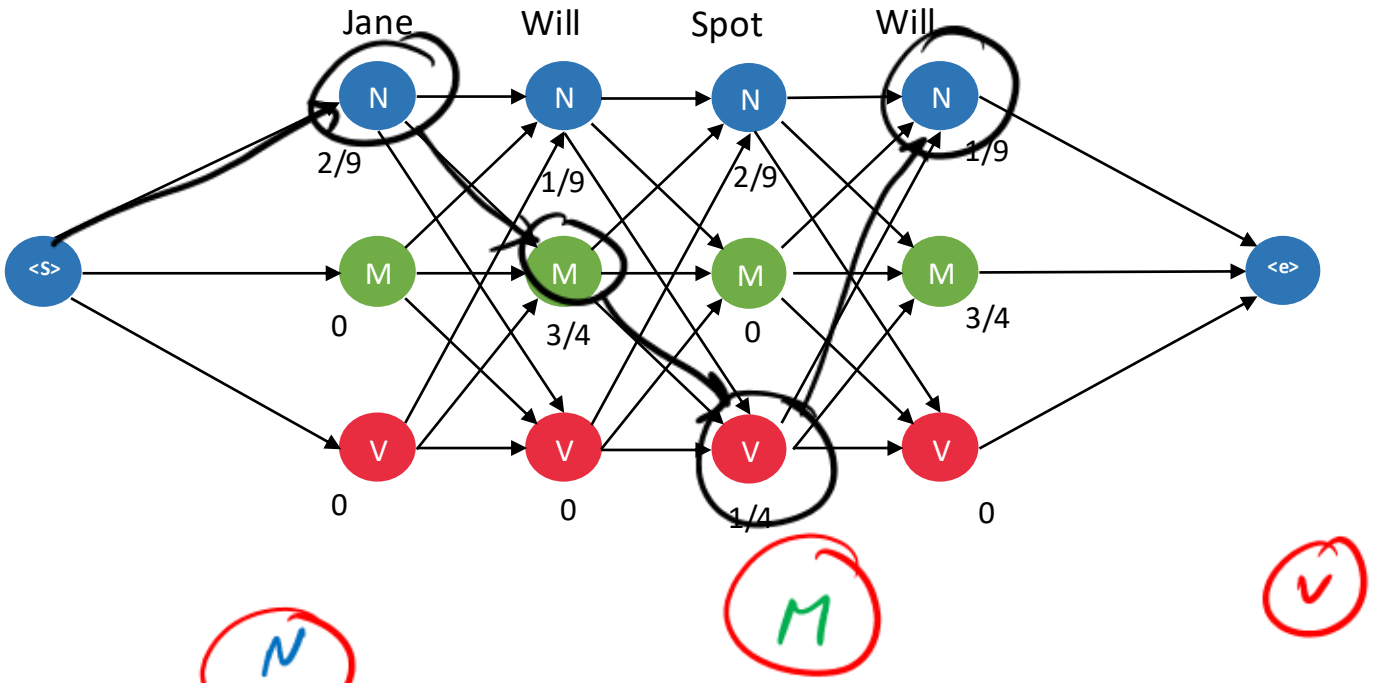
upGrad

	N	M	V
Mary	4/9	0	0
Jane	2/9	0	0
Will	1/9	3/4	0
Spot	2/9	0	1/4
Can	0	1/4	0
see	0	0	1/2
pat	0	0	1/4

Emission Probability Matrix

	N	M	V	<E>
<S>	3/4	1/4	0	0
N	1/9	1/3	1/9	4/9
M	¼	0	3/4	0
V	1	0	0	0

Transition Probability Matrix



$p(tag|word)$

PARSING https://www.nltk.org/book/ch08.html

upGrad

Parsing means associating tree structure to a sentence, given a grammar (often a context-free grammar)

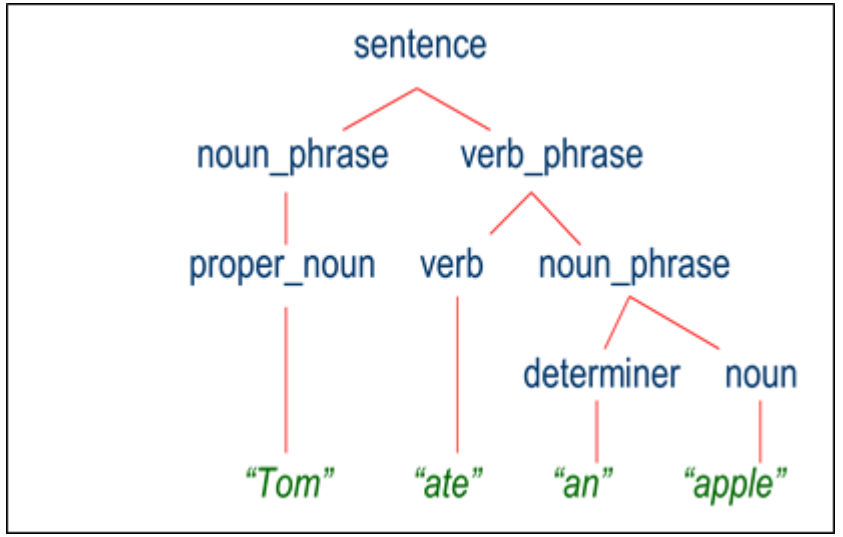
Algorithms:

A. Top-Down
   1. LL – Parses input from **L**eft-to-right, performing **L**eftmost derivation of the sentence
   2. Recursive Descent – Tail Recursive or Pratt Parser

B. Bottom-Up
   1. Precedence – Simple(recognizes the text's lowest-level small details first, then mid-level structures, and lastly highest-level overall structure), Shunting Yard
   2. Bounded-Context – LR (**L**eft-to-right, performing **R**ightmost derivation of the sentence)

A good parser shall have the ability to handle:

1. Prepositional Phrase attachment (e.g. – *I saw a man with a telescope*)
2. Coordination Scope
3. Gerunds vs Adjectives
4. Question Answering
5. Machine Translation (subject-verb-object or subject-object-verb)
6. Information Extraction
7. Speech understanding and Interpretation

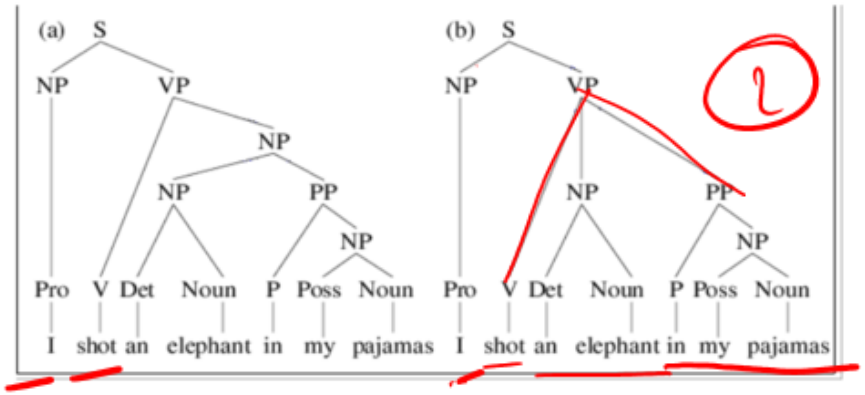


Probabilistic CFGs

upGrad

VP -> V NP ; NP -> NP PP
VP -> V NP PP

"I shot an elephant in my pajamas"



The rules of a **CFG** can lead us to multiple parses of the same sentence, leading to ambiguity over which parse is the correct one for the sentence. Consider the sentence:

"I shot an elephant in my pajamas."

(a) "I shot an elephant while I was wearing my pajamas."
(b) "I shot an elephant that was wearing my pajamas."

Obviously the second parse is nonsensical and we would like to pick the first parse as the proper one for the sentence PCFGs allow us to disambiguate by calculating which parse tree is the most probable.

Probabilistic Context-Free Grammars (PCFGs) are used when we want to find the most probable parsed structure of the sentence. PCFGs are grammar rules, similar to what you have seen, along with probabilities associated with each production rule.

Deriving a PCFG from a Corpus

upGrad

Having defined PCFGs, the next question is the following: how do we derive a PCFG from a corpus? We will assume a set of training data, which is simply a set 9 of parse trees t1, t2, . . . , tm

Each parse tree ti is a sequence of context-free rules: we assume that every parse tree in our corpus has the same symbol, S, at its root. We can then define a PCFG (N, Σ, S, R, q) as follows:

- N is the set of all non-terminals seen in the trees $t_1 \ldots t_m$.
- Σ is the set of all words seen in the trees $t_1 \ldots t_m$.
- The start symbol S is taken to be S.
- The set of rules R is taken to be the set of all rules $\alpha \rightarrow \beta$ seen in the trees $t_1 \ldots t_m$.
- The maximum-likelihood parameter estimates are

$$q_{ML}(\alpha \rightarrow \beta) = \frac{\text{Count}(\alpha \rightarrow \beta)}{\text{Count}(\alpha)}$$

where $\text{Count}(\alpha \rightarrow \beta)$ is the number of times that the rule $\alpha \rightarrow \beta$ is seen in the trees $t_1 \ldots t_m$, and $\text{Count}(\alpha)$ is the number of times the non-terminal α is seen in the trees $t_1 \ldots t_m$.

For example, if the rule VP → Vt NP is seen 105 times in our corpus, and the non-terminal VP is seen 1000 times, then

$$q(\text{VP} \rightarrow VP\ NP) = \frac{105}{1000}$$

Poll

Given the Grammar which tree seems more probable ?

S -> NP VP [1.0]
PP -> P NP [1.0]
VP -> V NP [0.9] | VP PP [0.1]
NP -> NP PP [0.4]
P -> 'with' [1.0]
V -> 'saw' [1.0]
NP -> 'astronomer' [0.1] | 'ears' [0.18] | 'saw' [0.04] | 'stars' [0.18] | 'telescope' [0.1]

NP -> NP PP (0.4)

A
(S
(NP astronomer)
(VP (V saw) (NP (NP stars) (PP (P with) (NP telescope)))))

B
(S
(NP astronomer)
(VP (VP (V saw) (NP stars)) (PP (P with) (NP telescope))))

VP -> VP PP (0.1)

Poll (Answer)

upGrad

Given the Grammar which tree seems more probable ?

S -> NP VP [1.0]
PP -> P NP [1.0]
VP -> V NP [0.9] | VP PP [0.1]
NP -> NP PP [0.4]
P -> 'with' [1.0]
V -> 'saw' [1.0]
NP -> 'astronomer' [0.1] | 'ears' [0.18] | 'saw' [0.04] | 'stars' [0.18] | 'telescope' [0.1]

A

(S
(NP astronomer)
(VP (V saw) (NP (NP stars) (PP (P with) (NP telescope)))))

(P = 0.000648)

B

(S
(NP astronomer)
(VP (VP (V saw) (NP stars)) (PP (P with) (NP telescope))))

(P = 0.000162)

Chomsky Normal Form

upGrad

Chomsky Normal Form. A grammar where every production is either of the form

$A \rightarrow BC$ (can't be S) or $A \rightarrow c$

(where A, B, C are arbitrary variables and c an arbitrary symbol).

The key advantage is that in Chomsky Normal Form, every derivation of a string of n letters has exactly $2n - 1$ steps

Let n be the length of a string. We start with the (non-terminal) symbol S which has length n=1.

Using n−1 rules of form (non−terminal)→(non−terminal)(non−terminal) we can construct a string containing n non-terminal symbols.

Then on each non-terminal symbol of said string of length n we apply a rule of form (non−terminal)→(terminal). i.e. we apply n rules.

In total we will have applied n−1+n=2n−1 rules.

Practice :

https://www.cs.bgu.ac.il/~auto202/wiki.files/9a.pdf

https://www.youtube.com/watch?v=IiCbNhHwsws

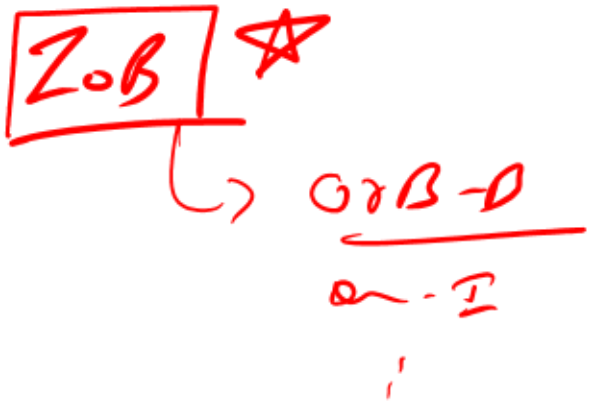
INFORMATION EXTRACTION

upGrad

Information Extraction (IE) system that can extract structured data from unstructured textual data. A key component in information extraction systems is **Named-Entity-Recognition (NER)**.

Assuming we are trying to make a conversational flight-booking system, which can show relevant flights when given a natural-language query such as "Please show me all morning flights from Bangalore to Mumbai on next Monday." For the system to be able to process this query, it has to extract useful **named entities** from the unstructured text query and convert them to a structured format, such as the following dictionary/JSON object:

```
{
    source: 'Bangalore',
    destination: 'Mumbai',
    day: 'Monday',
    time-of-day: 'morning'
}
```



Using these entities, we could query a database and get all relevant flight results. In general, named entities refer to names of people, organizations (Google), places (India, Mumbai), specific dates and time (Monday, 8 pm) etc.

INFORMATION EXTRACTION

upGrad

CRFs are used in a wide variety of sequence labelling tasks across various domains - POS tagging, speech recognition, NER etc

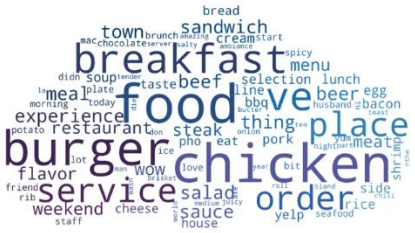
It is an **discriminative probabilistic classifiers** thus given a good data volume it out performs generative classifiers like HMMs.

CRFs use 'feature functions' rather than the input word sequence **x** itself. The idea is similar to how we add features for building a simple classification models like naive Bayes, decision tree classifiers etc.

- Word and POS tag based features:
  - word_is_city, word_is_digit, is_numeric,is_title etc.
- Label-based features: previous_label

CRFs can naturally consider state-to-state dependencies and feature-to-state dependencies. On the other hand, basic classification models do not consider such dependencies.

upGrad



SEMANTIC PROCESSING

Semantic processing

upGrad

Semantic processing is about understanding the meaning of a given piece of text.

But what do we mean by 'understanding the meaning' of text?

Example

"The ***bank*** will not be accepting cash on Saturdays. "

"The river overflowed the ***bank.***"

The word ***bank*** in the first sentence refers to the commercial (finance) banks, while in second sentence, it refers to the river bank.

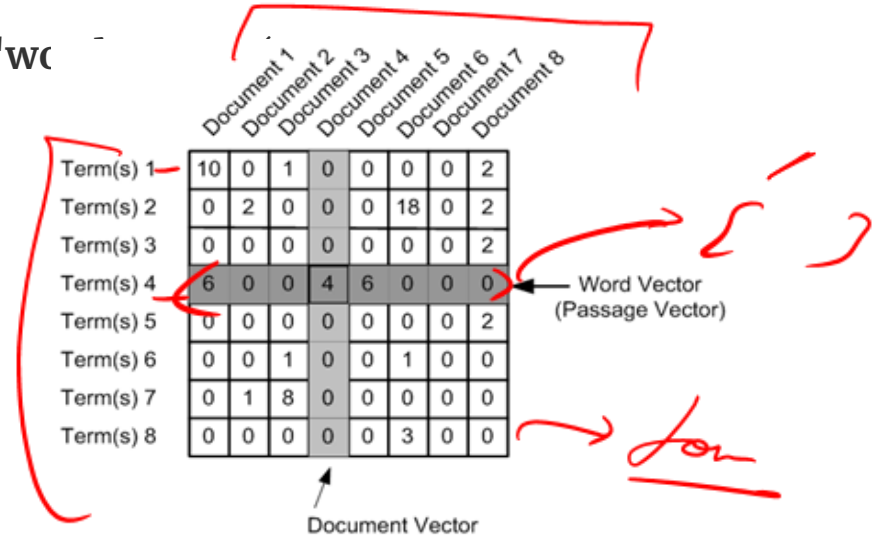
Our brain can process sentences meaningfully because it can relate the text to other words and concepts it already knows.

"words which appear in the same contexts have similar meanings"

To exploit this property, we need to represent words in a format which encapsulates its similarity with other words.

The most commonly used representation of words is using 'wo

The term-document **occurrence matrix**, where each row is a term in the vocabulary and each column is a document (such as a webpage, tweet, book etc.)



$$X = \begin{array}{c} \\ I \\ like \\ enjoy \\ deep \\ learning \\ NLP \\ flying \\ . \end{array} \begin{array}{c} \begin{array}{cccccccc} I & like & enjoy & deep & learning & NLP & flying & . \end{array} \\ \begin{bmatrix} 0 & 2 & 1 & 0 & 0 & 0 & 0 & 0 \\ 2 & 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 \end{bmatrix} \end{array}$$

The term-term **co-occurrence matrix**, where the ith row and jth column represents the occurrence of the ith word *in the context of* the jth word.

co-occurrence matrix

upGrad

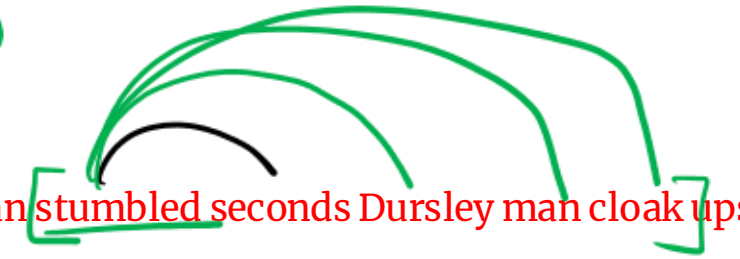
There are two ways of creating a co-occurrence matrix:

Using the occurrence context (e.g. a sentence):

Each sentence is represented as a context (there can be other definitions as well). If two terms occur in the same context, they are said to have occurred in the same occurrence context.

Skip-grams (x-skip-n-grams):

A sliding window will include the (x+n) words. This window will serve as the context now. Terms that co-occur within this context are said to have co-occurred.



"Man stumbled seconds Dursley man cloak upset knocked ground"

(Man, stumbled) (Man, seconds) (Man, Dursley) (Man, man)
(stumbled, seconds) (stumbled, Dursley) (stumbled, man) (stumbled, cloak)
(seconds, Dursley) (seconds, man) (seconds, cloak) (seconds, upset)
(Dursley, man) (Dursley, cloak) (Dursley, upset) (Dursley, knocked)
(man, cloak) (man, upset) (man, knocked) (man, ground)
(cloak, upset) (cloak, knocked) (cloak, ground)
(upset, knocked) (upset, ground)
(knocked, ground)

Problems with the text feature vectorizers like count vectorizers and TFIDF vectorizers are:

- High Dimensionality
- Limited vocabulary

Word Embeddings are a compressed, **low dimensional** version of the mammoth-sized text feature vectors

- **What are the different ways we can generate Word Embeddings ?**
  - **Frequency-based approach:** Reducing the term-document matrix (which can as well be a TF-IDF, incidence matrix etc.) using a dimensionality reduction technique such as SVD
  - **Prediction based approach:** In this approach, the input is a single word (or a combination of words) and output is a combination of context words (or a single word). A shallow neural network learns the embedding's such that the output words can be predicted using the input words.

TRAINING

There are two popular methods that we can use to train our embedding's weights: skip-gram and continuous bag of words (CBOW).

SKIP-GRAM

The idea behind skip-gram is to take in a word and predict the context words.

E.g. lets generate training data from a sentence like "the quick brown fox jumped over the lazy dog"

We need to predict context words from a target word where the context words are the words to the left and to the right of the target word. We will choose a **window size** (1 in this case) to decide how far left and right to go. The training data for the sentence above will look like this:

(quick, the),(quick, brown),(brown, quick),(brown, fox) …

The idea is to be able to train our weights so that we predict context words from target words. So when we input 'quick', we should receive 'the'. We do this across the entire training set because we want to adjust the weights for all of our words using all the context we have available.

CBOW

The continuous bag of words method allows us to create the training data for adjusting our embedding weights in a different way. Instead of predicting context words from target, we will feed in context words and try to predict the target word. So now the training data (with window of 1) will look something like

(['the' , 'brown'] , 'quick')

where we sum the embeddings of 'the' and 'brown' and expect to predict 'quick'.

COMPARISON

Though CBOW (predict target from context) and skip-gram (predict context words from target) are just inverted methods to each other, each have their advantages/disadvantage.

Since CBOW can use many context words to predict the target word, it acts like regularization and offers very good performance when our input data is not so large.
However the skip-gram model is more **fine grained** so we are able to extract more information and essentially have more accurate embeddings when we have a large data set

(Large data is the best regularizer)

Other Word Embeddings

After the success of **Word2Vec**, several other effective word embedding techniques have been developed by various teams. One of the most popular is **GloVe (Global Vectors for Words)** developed by a Stanford research group. Another recently developed and fast-growing word embedding library is fastText developed by Facebook AI Research (FAIR).

These embeddings are trained on billions of words (i.e. unique tokens), and thankfully, are available as pre-trained word vectors ready to use for text applications (for free!).

Additional Readings

Word2Vec: Original paper of word2vec by Mikolov et al.
GloVe vectors: Homepage (contains downloadable trained GloVe vectors, training methodology etc.).
fastText: Word embeddings developed by FAIR on multiple languages, available for download here

Appendix- Topic wise Reading Reference

Topics	Reading Reference
Regex	https://regexone.com
Bag of Words Model	https://machinelearningmastery.com/gentle-introduction-bag-words-model/
TF-IDF	https://stevenloria.com/tf-idf/ , http://blog.christianperone.com/2011/09/machine-learning-text-feature-extraction-tf-idf-part-i/ , http://aimotion.blogspot.com/2011/12/machine-learning-with-python-meeting-tf.html
Stemming and Lemmatization	https://nlp.stanford.edu/IR-book/html/htmledition/stemming-and-lemmatization-1.html
POS- Tagging	https://nlpforhackers.io/training-pos-tagger/ , https://pythonprogramming.net/part-of-speech-tagging-nltk-tutorial/ , https://towardsdatascience.com/part-of-speech-tagging-with-hidden-markov-chain-models-e9fccc835c0e ,
Parsing	https://www.commonlounge.com/discussion/1ef22e14f82444b090c8368f9444ba78
Named Entity Recognition	https://www.commonlounge.com/discussion/d5abf44e9d09490b8653dd75feff760e
Word Net	https://pythonprogramming.net/wordnet-nltk-tutorial/
Concept Net	http://conceptnet.io/ , http://tomkdickinson.co.uk/2017/05/combining-wordnet-and-conceptnet-in-neo4j/
Topic Modelling	https://www.youtube.com/watch?v=BuMu-bdoVrU
Word2Vec	https://arxiv.org/pdf/1411.2738.pdf , http://mccormickml.com/2016/04/19/word2vec-tutorial-the-skip-gram-model/ https://www.youtube.com/watch?v=TsEGsdVJjuA

Appendix- NLP Tutorials

POPULAR VIDEOS AND ARTICLES	
Text mining (NLP) with Python – Implementation Tutorial	https://nbviewer.jupyter.org/github/TiesdeKok/Python_NLP_Tutorial/blob/master/NLP_Notebook.ipynb
NLP Workshop – Concrete Solution to Real World Problem	https://github.com/hundredblocks/concrete_NLP_tutorial
NLP With Deep Learning – Stanford School OF Engineering	https://www.youtube.com/watch?v=OQQ-W_63UgQ&list=PL3FW7Lu3i5Jsnh1rnUwq_TcylNr7EkRe6
Compilation of 150 NLP and Python Tutorial	https://medium.com/machine-learning-in-practice/over-150-of-the-best-machine-learning-nlp-and-python-tutorials-ive-found-ffce2939bd78
Step by Step Guide	https://blog.insightdatascience.com/how-to-solve-90-of-nlp-problems-a-step-by-step-guide-fda605278e4e
Ultimate Guide to Understand and Implement NLP	https://www.analyticsvidhya.com/blog/2017/01/ultimate-guide-to-understand-implement-natural-language-processing-codes-in-python/

https://paperswithcode.com/area/natural-language-processing
http://nlpprogress.com/

upGrad

#LifeKoKaroLift

Thank You!